

1. Write a shell command that checks whether the file /etc/passwd exists and prints "yes" or "no".

```
GNU nano 6.2      che
#!/bin/bash

if [ -f /etc/passwd ]; then
echo "yes"
else
echo "no"
fi
```

```
pgcp-esd@pc:~$ nano check_file_path.sh
pgcp-esd@pc:~$ chmod a+x check_file_path.sh
pgcp-esd@pc:~$ ./check_file_path.sh
yes
pgcp-esd@pc:~$
```

2. Create a script that checks whether a given directory is writable and prints an appropriate message.

```
GNU nano 6.2      CHECK WRIT
#!/bin/bash

dir="$1"
if [ -w "$dir" ]; then
echo "Directory $dir is writable"
else
echo "Directory $dir is not writable"
fi
```

```
pgcp-esd@pc:~/asgm4$ nano check_writable_directory.sh
pgcp-esd@pc:~/asgm4$ chmod a+x check_writable_directory.sh
pgcp-esd@pc:~/asgm4$ ./check_writable_directory.sh ../asgm4
Directory ../asgm4 is writable
pgcp-esd@pc:~/asgm4$
```

3. Using [ ], test whether a variable contains an empty string and print an appropriate message. is this correct

```
pgcp-esd@pc:~/asgm4$ var=""
pgcp-esd@pc:~/asgm4$ [ -z "$var" ] && echo "Variable is empty" || echo "Variable
is not empty"
Variable is empty
pgcp-esd@pc:~/asgm4$
```

4. Write an if statement that checks whether the current user is root.

```
#!/bin/bash

if [ "$EUID" -eq 0 ]; then
echo "You are root"
else
echo "You are not root"
fi
```

```
pgcp-esd@pc:~/asgm4$ nano check_root_user.sh
pgcp-esd@pc:~/asgm4$ chmod a+x check_root_user.sh
pgcp-esd@pc:~/asgm4$ ./check_root_user.sh
You are not root
pgcp-esd@pc:~/asgm4$ sudo ./check_root_user.sh
[sudo] password for pgcp-esd:
You are root
pgcp-esd@pc:~/asgm4$
```

5. Write a script that reads an integer and prints whether it is positive, negative, or zero.

```
#!/bin/bash

read -p "Enter an integer: " num
if [ "$num" -gt 0 ]; then
echo "Positive"
elif [ "$num" -lt 0 ]; then
echo "Negative"
else
echo "Zero"
fi
```

```
pgcp-esd@pc:~/asgm4$ nano check_interger_sign.sh
pgcp-esd@pc:~/asgm4$ chmod a+x check_interger_sign.sh
pgcp-esd@pc:~/asgm4$ ./check_interger_sign.sh
Enter an integer: -20
Negative
pgcp-esd@pc:~/asgm4$ ./check_interger_sign.sh
Enter an integer: 1
Positive
pgcp-esd@pc:~/asgm4$ ./check_interger_sign.sh
Enter an integer: 0
Zero
```

6. Create a script that accepts two strings as arguments and checks whether they are equal.

```
#!/bin/bash

s1="$1"
s2="$2"

if [ "$1" = "$2" ]; then
echo "Strings are equal"
else
echo "Strings are different"
fi
```

```
pgcp-esd@pc:~/asgm4$ nano cmp_two_string.sh
pgcp-esd@pc:~/asgm4$ chmod a+x cmp_two_string.sh
pgcp-esd@pc:~/asgm4$ ./cmp_two_string.sh Hello Hello
Strings are equal
pgcp-esd@pc:~/asgm4$ ./cmp_two_string.sh Hello hello
Strings are different
pgcp-esd@pc:~/asgm4$
```

7. Use test or [ ] to check whether /tmp is a directory.

```
pgcp-esd@pc:~/asgm4$ [ -d /tmp ] && echo "/tmp is a directory" || echo "/tmp is not a directory"
/tmp is a directory
pgcp-esd@pc:~/asgm4$
```

8. Write a script that checks whether a given filename is a regular file, directory, or does not exist.

```
#!/bin/bash

target="$1"
if [ -f "$target" ]; then
echo "$target is a regular file"
elif [ -d "$target" ]; then
echo "$target is a directory"
else
echo "$target doesn't exists"
fi
```

```
pgcp-esd@pc:~/asgm4$ nano check_file_exist.sh
pgcp-esd@pc:~/asgm4$ chmod a+x check_file_exist.sh
pgcp-esd@pc:~/asgm4$ ./check_file_exist.sh cmp_two_string.sh
cmp_two_string.sh is a regular file
```

9. Write a for loop that prints the names of all .txt files in the current directory.

```
GNU nano 8.7.1                                print_curr_dir.sh
#!/bin/bash

for file in *.txt; do
    [ -e "$file" ] && echo "$file"
done
```

```
workspace@ubuntu:~/asgmt$ nano print_curr_dir.sh
workspace@ubuntu:~/asgmt$ chmod a+x print_curr_dir.sh
workspace@ubuntu:~/asgmt$ cd ..
workspace@ubuntu:~$ ./asgmt/print_curr_dir.sh
file1.txt
file.txt
workspace@ubuntu:~$
```

10. Write a for loop that prints the numbers from 1 to 20.

```
GNU nano 8.7.1
#!/bin/bash

for i in {1..20}; do
    echo "$i"
done
```

```
workspace@ubuntu:~/workspace/asgmt$ nano forloop_print_num.sh
workspace@ubuntu:~/workspace/asgmt$ chmod a+x forloop_print_num.sh
workspace@ubuntu:~/workspace/asgmt$ ./forloop_print_num.sh
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
workspace@ubuntu:~/workspace/asgmt$
```

11. Write a while loop that prints the numbers from 1 to 10.

```
#!/bin/bash

i=1
while [ $i -le 10 ]; do
echo "$i"
((i++))
done
```

```
workspace@ubuntu:~/asgmt$ nano while_print_num.sh
workspace@ubuntu:~/asgmt$ chmod a+x while_print_num.sh
workspace@ubuntu:~/asgmt$ ./while_print_num.sh
1
2
3
4
5
6
7
8
9
10
```

12. Write an until loop that keeps asking the user for a password until the correct password is entered.

```
#!/bin/bash

correct_pass="secret123"
pass=""

until [ "$pass" = "$correct_pass" ]; do
read -s -p "Enter The Password: " pass
echo
done
echo "Access Granted"
```

```
workspace@ubuntu:~/asgmt$ nano until_correct_passwd.sh
workspace@ubuntu:~/asgmt$ ./until_correct_passwd.sh
Enter The Password:
Enter The Password:
Enter The Password:
Access Granted
```

13. Write a script using a for loop to calculate the sum of numbers from 1 to N.

```
#!/bin/bash

n=$1
sum=0
for ((i=1; i<=n; i++)); do
sum=$((sum + i))
done
echo "Sum of 1 to $n is: $sum"
```

```
workspace@ubuntu:~/asgmt$ nano for_sum_toN.sh
workspace@ubuntu:~/asgmt$ ./for_sum_toN.sh 10
Sum of 1 to 10 is: 55
workspace@ubuntu:~/asgmt$
```

14. Write a script to calculate the factorial of a given number using a loop.

```
#!/bin/bash

n=$1
fact=1
for (( i=1; i<=n; i++ )); do
fact=$((fact * i))
done
echo "Factorial of $n is $fact"
```

```
workspace@ubuntu:~/asgmt$ nano for_fact_ofN.sh
workspace@ubuntu:~/asgmt$ chmod a+x for_fact_ofN.sh
workspace@ubuntu:~/asgmt$ ./for_fact_ofN.sh
Factorial of 1 is 1
workspace@ubuntu:~/asgmt$ ./for_fact_ofN.sh 5
Factorial of 5 is 120
workspace@ubuntu:~/asgmt$
```

15. Write a script that counts the number of files in a given directory using a loop.

```
#!/bin/bash

count=0

for i in "$1"/*
do
if [ -f $i ]; then
count=$((count+1))
fi
done
echo "Number of files in $1 is $count"
```

```
workspace@ubuntu:~/asgmt$ nano count_files_in_anydir.sh
workspace@ubuntu:~/asgmt$ ./count_files_in_anydir.sh ~/asgmt
Number of files in /home/workspace/asgmt is 8
workspace@ubuntu:~/asgmt$
```

16. Write a script that prints all even numbers between 1 and 100 using a loop.

```
#!/bin/bash

for i in {1..100}; do
cond=$((i%2))
if [ $cond -eq 0 ]; then
echo "$i"
fi
done
```

```
workspace@ubuntu:~/asgmt$ nano loop_print_evennum.sh
workspace@ubuntu:~/asgmt$ ./loop_print_evennum.sh
2
4
6
8
10
12
14
```

17. Write a script that checks each number from 1 to N and prints whether it is even or odd.

```
#!/bin/bash

n=$1
for (( i=1; i<=n; i++ )); do
if [  $((i\%2))$  -eq 0 ]; then
echo "$i is Even"
else
echo "$i is Odd"
fi
done
```

```
workspace@ubuntu:~/asgmt$ nano print_odd_even_toN.sh
workspace@ubuntu:~/asgmt$ chmod a+x print_odd_even_toN.sh
workspace@ubuntu:~/asgmt$ ./print_odd_even_toN.sh _
workspace@ubuntu:~/asgmt$ ./print_odd_even_toN.sh 10
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
workspace@ubuntu:~/asgmt$
```

18. Write a script that accepts a filename and counts how many lines in the file are non-empty.

```
GNU nano 8.7.1 count_
#!/bin/bash

fn=$1

count=0

while read line
do
if [ -n "$line" ]
then
count=$((count+1))
fi
done <"$1"
echo "Non empty lines are $count"
```

```
workspace@ubuntu:~/asgmt$ nano count_lines_infile.sh
workspace@ubuntu:~/asgmt$ chmod a+x count_lines_infile.sh
workspace@ubuntu:~/asgmt$ ./count_lines_infile.sh
./count_lines_infile.sh: line 13: : No such file or directory
Non empty lines are 0
workspace@ubuntu:~/asgmt$ ./count_lines_infile.sh print_odd_even_toN.sh
Non empty lines are 9
workspace@ubuntu:~/asgmt$
```

19. Write a shell command to start sleep 100 in the background and display the running job.

```
workspace@ubuntu:~/asgmt$ sleep 100 &
[1] 14381
workspace@ubuntu:~/asgmt$ jobs
[1]+  Running                  sleep 100 &
```

20. Start a long-running command in the background and demonstrate how to use jobs to display it.

```
workspace@ubuntu:~/asgmt$ sleep 100 &
[1] 14381
workspace@ubuntu:~/asgmt$ jobs
[1]+  Running                  sleep 100 &
```

21. Start a background job and use fg to bring it to the foreground.

```
workspace@ubuntu:~/asgnmt$ jobs
[1]+  Running                  sleep 100 &
workspace@ubuntu:~/asgnmt$ fg %1
sleep 100
█
```

22. Stop a foreground job using Ctrl+Z, then use bg to continue it in the background.

```
workspace@ubuntu:~/asgnmt$ sleep 100
^Z
[1]+  Stopped                  sleep 100
workspace@ubuntu:~/asgnmt$ bg %1
[1]+ sleep 100 &
workspace@ubuntu:~/asgnmt$
```

23. Start two background sleep processes and use jobs to identify both jobs.

```
[1]+ sleep 100 &
workspace@ubuntu:~/asgnmt$ sleep 90 &
[2] 14503
workspace@ubuntu:~/asgnmt$ jobs
[1]-  Running                  sleep 100 &
[2]+  Running                  sleep 90 &
workspace@ubuntu:~/asgnmt$ █
```

24. Start a background process, find its PID using ps, and terminate it using kill.

```
[1]+  Done                    sleep 50
workspace@ubuntu:~/asgnmt$ sleep 500 &
[1] 14523
workspace@ubuntu:~/asgnmt$ ps
  PID TTY          TIME CMD
 12242 pts/0    00:00:00 bash
  14523 pts/0    00:00:00 sleep
  14525 pts/0    00:00:00 ps
workspace@ubuntu:~/asgnmt$ ps aux | grep "sleep 500"
workspa+  14523  0.0  0.1 16112  7872 pts/0    S   13:16   0:00 sleep 500
workspa+  14532  0.0  0.0  9316  2676 pts/0    S+  13:16   0:00 grep --color=aut
o sleep 500
workspace@ubuntu:~/asgnmt$ kill 14523
[1]+  Terminated              sleep 500
```

25. Start a process and send it SIGTERM using kill.

```
workspace@ubuntu:~/asgmt$ sleep 100 &
[1] 4891
workspace@ubuntu:~/asgmt$ kill -15 4891
[1]+  Terminated                  sleep 100
workspace@ubuntu:~/asgmt$
```

26. Start a process and demonstrate the difference between kill PID and kill -9 PID.

```
workspace@ubuntu:~/asgmt$ sleep 1000 &
[1] 5116
workspace@ubuntu:~/asgmt$ ps
  PID TTY          TIME CMD
 4295 pts/0        00:00:00 bash
 5116 pts/0        00:00:00 sleep
 5118 pts/0        00:00:00 ps
workspace@ubuntu:~/asgmt$ kill 5116
[1]+  Terminated                  sleep 1000
workspace@ubuntu:~/asgmt$ kill -9 5116
bash: kill: (5116) - No such process
workspace@ubuntu:~/asgmt$ sleep 1000 &
[1] 5123
workspace@ubuntu:~/asgmt$ kill -9 5123
[1]+  Killed                      sleep 1000
workspace@ubuntu:~/asgmt$
```

27. Use ps to display all processes belonging to the current user.

```
[1]+  Killed                      sleep 1000
workspace@ubuntu:~/asgmt$ ps -u $USER
  PID TTY          TIME CMD
 2794 ?            00:00:01 systemd
 2805 ?            00:00:00 (sd-pam)
 2826 ?            00:00:01 dbus-daemon
 2827 ?            00:00:01 pipewire
 2831 ?            00:00:00 gnome-keyring-d
 2832 ?            00:00:00 mpris-proxy
 2833 ?            00:00:00 wireplumber
 2835 ?            00:00:00 pipewire
 2836 ?            00:00:00 pipewire-pulse
 2902 ?            00:00:00 xdg-document-po
 2904 tty2        00:00:00 gdm-wayland-ses
 2925 tty2        00:00:00 gnome-session-i
 2934 ?            00:00:00 xdg-permission-
 3083 ?            00:00:00 gcr-ssh-agent
 3084 ?            00:00:00 gnome-session-c
```

28. Use ps to display PID, PPID, CPU usage, memory usage, and command name.

```
workspace@ubuntu:~/asgmt$ ps -eo pid,ppid,%cpu,%mem,comm
  PID      PPID %CPU %MEM COMMAND
    1         0  0.6  0.3 systemd
    2         0  0.0  0.0 kthreadd
    3         2  0.0  0.0 pool_workqueue_release
    4         2  0.0  0.0 kworker/R-rcu_gp
    5         2  0.0  0.0 kworker/R-sync_wq
    6         2  0.0  0.0 kworker/R-kvfree_rcu_reclaim
    7         2  0.0  0.0 kworker/R-slab_flushwq
    8         2  0.0  0.0 kworker/R-netns
   10         2  0.0  0.0 kworker/0:0H-kblockd
```

29. Use ps and grep to find whether a particular process is running.

```
workspace@ubuntu:~/asgmt$ ps aux | grep brave
workspa+  4282  0.0  0.0 10136 3924 ?        S    13:54   0:00 /bin/bash /u
sr/bin/brave-browser-stable
workspa+  4289  7.1  6.7 55709592 373044 ?      Sl   13:54   1:09 /opt/brave.c
om/brave/brave
workspa+  4325  0.0  0.0 54550372 4148 ?        Sl   13:54   0:00 /opt/brave.c
om/brave/chrome_crashpad_handler --monitor-self --monitor-self-annotation=ptype
=crashpad-handler --database=/home/workspace/.config/BraveSoftware/Brave-Browse
r/Crash Reports --metrics-dir=/home/workspace/.config/BraveSoftware/Brave-Brows
er --url=https://cr.brave.com --annotation=lsb-release=Ubuntu 26.04.1 LTS --ann
otation=plat=Linux --annotation=prod=Chrome_Linux --annotation=ver=153.1.95.101
--initial-client-fd=5 --shared-client-connection
```

30. Write a command to count how many processes are currently running on the system.

```
workspace@ubuntu:~/asgmt$ ps aux | wc -l
303
workspace@ubuntu:~/asgmt$
```

31. Use ps and sort to display the top 5 processes by CPU usage.

```
workspace@ubuntu:~/asgmt$ ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -6
  PID %CPU %MEM COMMAND
 5286  400  0.0 ps
 4563  15.2  8.2 brave
 3124  14.0  5.2 gnome-shell
 4476   7.3  7.4 brave
 4363   6.2  3.6 brave
```

32. Use ps and sort to display the top 5 processes by memory usage.

```
workspace@ubuntu:~/asgmt$ ps -eo pid,%cpu,%mem,comm --sort=-%mem | head -6
  PID %CPU %MEM    COMMAND
  4476   7.2   8.5    brave
  4507   4.8   7.0    brave
  4563  14.0   7.0    brave
  4289   6.1   7.0    brave
  4587   2.4   5.2    brave
```

33. Use ps and grep to find all processes whose command contains the word bash.

```
workspace@ubuntu:~/asgmt$ ps aux | grep '[b]ash'
workspace+  4282  0.0  0.0  10136  3924 ?        S   13:54   0:00 /bin/bash /u
sr/bin/brave-browser-stable
workspace+  4295  0.0  0.1  11552  5992 pts/0    Ss  13:54   0:00 /usr/bin/bas
h
workspace@ubuntu:~/asgmt$
```

34. Write a script that checks every 5 seconds whether a specified process is running.

```
#!/bin/bash

while true
do
if pgrep "$1" > /dev/null
then
echo "$1 is running"
else
echo "$1 is not running"
fi

sleep 5
done
```

```
workspace@ubuntu:~/asgmt$ nano check_ps_pid.sh
workspace@ubuntu:~/asgmt$ chmod a+x check_ps_pid.sh
workspace@ubuntu:~/asgmt$ ./check_ps_pid.sh bash
bash is running
bash is running
bash is running
^C
workspace@ubuntu:~/asgmt$ sleep 15 &
[1] 5593
workspace@ubuntu:~/asgmt$ ./check_ps_pid.sh sleep
sleep is running
sleep is not running
```

35. Write a script that asks the user for a process name and displays matching processes using ps and grep.

```
#!/bin/bash

read -p "Enter the process name: " process

if pgrep "$process" > /dev/null
then
ps aux | grep "$process"
else
echo "No such process"
fi
```

```
workspace@ubuntu:~/asgmt$ nano check_ps.sh
workspace@ubuntu:~/asgmt$ ./check_ps.sh
Enter the process name: bash
workspa+  4282  0.0  0.0  10136  3924 ?        S    13:54   0:00 /bin/bash /u
sr/bin/brave-browser-stable
workspa+  4295  0.0  0.1  11552  5992 pts/0    Ss   13:54   0:00 /usr/bin/bas
h
workspa+  5754  0.0  0.0  10136  3936 pts/0    S+   14:41   0:00 /bin/bash ./
check_ps.sh
workspa+  5757  0.0  0.0   9316  2608 pts/0    S+   14:41   0:00 grep bash
workspace@ubuntu:~/asgmt$
```

36. Write a script that accepts a filename and prints whether the file is readable, writable, and executable.

```
#!/bin/bash

file="$1"

if [ -r "$file" ]
then
echo "Readable"
else
echo "Not Readable"
fi

if [ -w "$file" ]
then
echo "Writable"
else
echo "Not Writable"
fi

if [ -x "$file" ]
then
echo "Executable"
else
echo "Not Executable"
fi
```

```
workspace@ubuntu:~/asgmt$ ls -l test.txt
-rwxrwx--- 1 workspace workspace 0 Sep 13 14:46 test.txt
workspace@ubuntu:~/asgmt$ ./check_file_access.sh test.txt
Readable
Writable
Executable
workspace@ubuntu:~/asgmt$ nano check_file_access.sh
workspace@ubuntu:~/asgmt$ ls -l test.txt
-rw----- 1 workspace workspace 0 Sep 13 14:46 test.txt
workspace@ubuntu:~/asgmt$ ./check_file_access.sh test.txt
Readable
Writable
Not Executable
```

37. Create a script that accepts multiple filenames as arguments and reports whether each file exists.

```
#!/bin/bash

for file in "$@"
do
if [ -e "$file" ]
then
echo "$file exists"
else
echo "$file doesn't exists"
fi
done
```

```
workspace@ubuntu:~/asgmt$ nano check_all_exists.sh
workspace@ubuntu:~/asgmt$ cd ..
workspace@ubuntu:~$ ./asgmt/check_all_exists.sh file.txt file1.txt file2.txt
file.txt exists
file1.txt exists
file2.txt doesn't exists
workspace@ubuntu:~$
```

38. Write a script that accepts a directory name as an argument and displays all files larger than 1 MB.

```
#!/bin/bash

dir="$1"
find "$dir" -type f -size +1M
```

```
workspace@ubuntu:~/asgmt$ nano display_1MB_files.sh
workspace@ubuntu:~/asgmt$ chmod a+x display_1MB_files.sh
workspace@ubuntu:~/asgmt$ ./display_1MB_files.sh /home/workspace
/home/workspace/.cache/BraveSoftware/Brave-Browser/Default/Cache/Cache_Data/822ef149c80f16aa_0
/home/workspace/.cache/BraveSoftware/Brave-Browser/Default/Cache/Cache_Data/1f5d7f752d97c182_0
/home/workspace/.cache/BraveSoftware/Brave-Browser/Default/Cache/Cache_Data/4ee40f2dc471637_0
/home/workspace/.cache/BraveSoftware/Brave-Browser/Default/Cache/Cache_Data/8f32c641cf025b77_0
```

39. Write a script that accepts a directory name and counts the number of directories inside it.

```
#!/bin/bash

count=0

for item in "$1"/*; do
if [ -d "$item" ]; then
count=$((count+1))
fi
done
echo "Number of Directories: $count"
```

```
workspace@ubuntu:~/asgmt$ nano count_dir.sh
workspace@ubuntu:~/asgmt$ chmod a+x count_dir.sh
workspace@ubuntu:~/asgmt$ ./count_dir.sh /home/workspace
Number of Directories: 10
```

40. Write a script that reads a file line by line and prints each line with its line number.

```
#!/bin/bash

count=1

while read line
do
echo "$count: $line"
count=$((count+1))
done < "$1"
```

```
workspace@ubuntu:~/asgmt$ nano print_line_with_count.sh
workspace@ubuntu:~/asgmt$ chmod a+x print_line_with_count.sh
workspace@ubuntu:~/asgmt$ ./print_line_with_count.sh notes.txt
1: Linux is an open-source operating system.
2: The command line interface is highly efficient.
3: Python and Bash are great for automation.
4: System administrators monitor logs daily.
5: Security is a top priority for servers.
```

41. Write a script that reads a file and counts the number of lines containing the word error using grep.

```
#!/bin/bash
```

```
grep -ic "error" "$1"
```

```
workspace@ubuntu:~/asgmt$ nano count_error.sh
workspace@ubuntu:~/asgmt$ chmod a+x count_error.sh
workspace@ubuntu:~/asgmt$ ./count_error.sh notes.txt
0
```

42. Write a command using grep to search for a word in a text file, ignoring case.

```
workspace@ubuntu:~/asgmt$ grep -i "linux" notes.txt
Linux is an open-source operating system.
```

43. Use grep to display only those lines from /etc/passwd that contain the word bash.

```
workspace@ubuntu:~/asgmt$ grep "bash" /etc/passwd
root:x:0:0:root:/root:/bin/bash
workspace:x:1000:1000:Pranshu:/home/workspace:/bin/bash
```

44. Use grep to search recursively for the word password in all files under a given directory.

```
workspace@ubuntu:~/asgmt$ grep -ri "linux" ./
./notes.txt:Linux is an open-source operating system.
```

45. Use grep with line numbers to display matching lines from a file.

```
workspace@ubuntu:~/asgmt$ grep -ni "linux" notes.txt
1:Linux is an open-source operating system.
```

46. Use grep to count how many lines in a file contain a given word.

```
workspace@ubuntu:~/asgmt$ grep -ci "linux" notes.txt
1
```

47. Write a command using sort to sort the contents of a text file alphabetically.

```
workspace@ubuntu:~/asgnmt$ sort notes.txt
Linux is an open-source operating system.
Python and Bash are great for automation.
Security is a top priority for servers.
System administrators monitor logs daily.
The command line interface is highly efficient.
```

48. Use sort -n to sort a file containing numbers in ascending order.

```
workspace@ubuntu:~/asgnmt$ sort -n numbers.txt
-10
0
7
45.99
555
12345
987654321
```

49. Use sort -r to sort a list in descending order.

```
workspace@ubuntu:~/asgnmt$ sort -r numbers.txt
987654321
7
555
45.99
12345
-10
0
workspace@ubuntu:~/asgnmt$ sort -r notes.txt
The command line interface is highly efficient.
System administrators monitor logs daily.
Security is a top priority for servers.
Python and Bash are great for automation.
Linux is an open-source operating system.
workspace@ubuntu:~/asgnmt$
```

50. Use sort together with uniq to display unique values from a file.

```
workspace@ubuntu:~/asgnmt$ sort repeated.txt | uniq
00
1
12345
242
Hello There
Hi
workspace@ubuntu:~/asgnmt$
```

51. Write a command to sort /etc/passwd based on the username field.

```
workspace@ubuntu:~/asgnmt$ sort -t: -k1 /etc/passwd
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
avahi:x:103:109:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
_chrony:x:988:988:Chrony Daemon:/var/lib/chrony:/usr/sbin/nologin
colord:x:977:977:colord colour management daemon:/var/lib/colord:/usr/s
gin
cups-browsed:x:110:111::/nonexistent:/usr/sbin/nologin
cups-ppk-helper:x:107:111:user for cups-ppk-helper service:/nonexistent:/
```

52. Create a script that accepts three numbers and prints the largest number using if-elif-else.

```
#!/bin/bash

read -p "Enter 3 Numbers: " a b c

if [ $a -gt $b ] && [ $a -gt $c ]
then
echo "$a is the largest"
elif [ $b -gt $a ] && [ $b -gt $c ]
then
echo "$b is the largest"
else
echo "$c is the largest"
fi
```

```
workspace@ubuntu:~/asgnmt$ nano largest.sh
workspace@ubuntu:~/asgnmt$ chmod a+x largest.sh
workspace@ubuntu:~/asgnmt$ ./largest.sh
Enter 3 Numbers: 10 20 -1
20 is the largest
workspace@ubuntu:~/asgnmt$
```

53. Write a script using case that accepts start, stop, and status and prints a corresponding message.

```
#!/bin/bash

read -p "Enter command: " choice
case $choice in
start)
echo "Starting process"
;;
stop)
echo "Stopping process"
;;
status)
echo "Checking status"
;;
*)
echo "Invalid option"
;;
esac
```

```
workspace@ubuntu:~/asgmt$ nano case_message.sh
workspace@ubuntu:~/asgmt$ chmod a+x case_message.sh
workspace@ubuntu:~/asgmt$ ./case_message.sh
Enter command: start
Starting process
workspace@ubuntu:~/asgmt$
```

54. Create a simple menu-driven script using case for options such as Create File, Delete File, Display File, and Exit.

```
#!/bin/bash

while true
do
echo "1. Create File"
echo "2. Delete File"
echo "3. Display File"
echo "4. Exit"

read -p "Enter choice: " choice

case $choice in
1)
read -p "Enter Filename: " file
touch "$file"
echo "$file created"
echo
;;

2)
read -p "Enter Filename: " file
rm "$file"
echo "$file removed"
echo
;;

3)
read -p "Enter Filename: " file
cat "$file"
echo
;;

4)
echo "Exiting..."
break
;;

*)
echo "Invalid choice"
echo
;;

esac
done
```

```
workspace@ubuntu:~/asgmt$ nano menu.sh
workspace@ubuntu:~/asgmt$ ./menu.sh
1. Create File
2. Delete File
3. Display File
4. Exit
Enter choice: 1
Enter Filename: hello.txt
hello.txt created

1. Create File
2. Delete File
3. Display File
4. Exit
Enter choice: 2
Enter Filename: hello.txt
hello.txt removed

1. Create File
2. Delete File
3. Display File
4. Exit
Enter choice: 4
Exiting...
workspace@ubuntu:~/asgmt$
```

55. Write a script that accepts a number and prints its multiplication table from 1 to 10.

```
#!/bin/bash

read -p "Enter number: " n

for i in {1..10}; do
echo "$n x $i = $((n*i))"
done
```

```
workspace@ubuntu:~/asgmt$ nano mul.sh
workspace@ubuntu:~/asgmt$ chmod a+x mul.sh
workspace@ubuntu:~/asgmt$ ./mul.sh
Enter number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

56. Write a script that accepts a string and determines whether it is a palindrome.

```
#!/bin/bash

read -p "Enter the string: " string

reverse=$(echo "$string" | rev)

if [ "$string" = "$reverse" ]
then
echo "Palindrome"
else
echo "Not a palindrome"
fi
```

```
workspace@ubuntu:~/asgnmt$ nano palindrome.sh
workspace@ubuntu:~/asgnmt$ chmod a+x palindrome.sh
workspace@ubuntu:~/asgnmt$ ./palindrome.sh
Enter the string: tenet
Palindrome
workspace@ubuntu:~/asgnmt$ ./palindrome.sh
Enter the string: raspberry
Not a palindrome
```

57. Write a script that counts the number of vowels in a string using a loop.

```
#!/bin/bash

read -p "Enter string: " str
count=0

for (( i=0; i<${#str}; i++ )); do
char=${str:i:1}
case $char in
a|e|i|o|u|A|E|I|O|U)
count=$((count+1))
;;
esac
done
echo "Number of vowels: $count"
```

```
workspace@ubuntu:~/asgnmt$ nano count_vowels.sh
workspace@ubuntu:~/asgnmt$ chmod a+x count_vowels.sh
workspace@ubuntu:~/asgnmt$ ./count_vowels.sh
Enter string: linux
Number of vowels: 2
```

58. Write a script that accepts a filename and displays its first 5 and last 5 lines.

```
#!/bin/bash

echo "First 5 lines: "
head -5 "$1"

echo "Last 5 lines: "
tail -5 "$1"
echo
```

```
workspace@ubuntu:~/asgnmt$ nano head_tail.sh
workspace@ubuntu:~/asgnmt$ ./head_tail.sh notes2.txt
First 5 lines:
1. Apple
2. Banana
3. Cherry
4. Date
5. Elderberry
Last 5 lines:
11. Mango
12. Nectarine
13. Orange
14. Papaya
15. Quince
```

59. Write a script that searches for a given word in a file and prints whether it was found.

```
#!/bin/bash

if grep -q "$1" "$2"
then
echo "Word found"
else
echo "Word not found"
fi
```

```
workspace@ubuntu:~/asgmt$ nano search_word.sh
workspace@ubuntu:~/asgmt$ chmod a+x search_word.sh
workspace@ubuntu:~/asgmt$ ./search_word.sh Apple notes2.txt
Word found
```

60. Write a script that accepts a filename and counts the number of words, lines, and characters in it.

```
#!/bin/bash

echo "lines: $(wc -l $1), words: $(wc -w $1), characters: $(wc -c $1)"
```

```
Word found
workspace@ubuntu:~/asgmt$ nano wc_file.sh
workspace@ubuntu:~/asgmt$ chmod a+x wc_file.sh
workspace@ubuntu:~/asgmt$ ./wc_file.sh notes2.txt
lines: 15 notes2.txt, words: 30 notes2.txt, characters: 154 notes2.txt
workspace@ubuntu:~/asgmt$
```

61. Write a script that displays the largest and smallest number from a list of numbers entered by the user.

```
#!/bin/bash

read -p "Enter numbers: " numbers

first=1

for n in $numbers
do
    if [ $first -eq 1 ]
    then
        largest=$n
        smallest=$n
        first=0
    else
        if [ $n -gt $largest ]
        then
            largest=$n
        fi
        if [ $n -lt $smallest ]
        then
            smallest=$n
        fi
    fi
done

echo "Largest is $largest"
echo "Smallest is $smallest"
```

```
workspace@ubuntu:~/asgnmt$ nano largest_smallest.sh
workspace@ubuntu:~/asgnmt$ ./largest_smallest.sh
Enter numbers: 1 2 3 4 5
Largest is 5
Smallest is 1
workspace@ubuntu:~/asgnmt$
```

62. Write a script that accepts a number and determines whether it is prime.

```
#!/bin/bash

read -p "Enter a number: " n

count=0
for (( i=1; i<=n; i++ ))
do
if [ $(n%i) -eq 0 ]
then
count=$((count+1))
fi
done

if [ $count -eq 2 ]
then
echo "Prime"
else
echo "Not prime"
fi
```

```
workspace@ubuntu:~/asgnmt$ nano prime.sh
workspace@ubuntu:~/asgnmt$ ./prime.sh
Enter a number: 3
Prime
workspace@ubuntu:~/asgnmt$ ./prime.sh
Enter a number: 4
Not prime
workspace@ubuntu:~/asgnmt$
```

63. Write a script that prints all prime numbers between 1 and N.

```
#!/bin/bash

read -p "Enter Number: " n
for ((i=2; i<=n; i++ ))
do
    count=0
    for ((j=1; j<=i; j++))
    do
        if [ $((i%j)) -eq 0 ]; then
            count=$((count+1))
        fi
    done
    if [ $count -eq 2 ]
    then
        echo "$i"
    fi
done
```

```
workspace@ubuntu:~/asgmt$ nano print_prime.sh
workspace@ubuntu:~/asgmt$ chmod a+x print_prime.sh
workspace@ubuntu:~/asgmt$ ./print_prime.sh
Enter Number: 5
2
3
5
workspace@ubuntu:~/asgmt$
```

64. Write a script that accepts a filename and creates a backup with the extension .bak.

```
#!/bin/bash

cp "$1" "$1.bak"
echo "Backup created: $1.bak"
```

```
workspace@ubuntu:~/asgmt$ nano create_backup.sh
workspace@ubuntu:~/asgmt$ ./create_backup.sh notes.txt
Backup created: notes.txt.bak
```

65. Write a script that checks whether a file has been modified today using appropriate file-test or command-line utilities.

```
#!/bin/bash

if [ "$(date -r "$1" +%Y-%m-%d)" = "$(date +%Y-%m-%d)" ]
then
echo "File was modified today"
else
echo "File was not modified today"
fi
```

```
workspace@ubuntu:~/asgmt$ nano file_modify.sh
workspace@ubuntu:~/asgmt$ chmod a+x file_modify.sh
workspace@ubuntu:~/asgmt$ ./file_modify.sh
date: '': No such file or directory
File was not modified today
workspace@ubuntu:~/asgmt$ ./file_modify.sh notes2.txt
File was modified today
workspace@ubuntu:~/asgmt$
```

66. Write a script that accepts a directory and deletes all .tmp files after asking the user for confirmation.

```
#!/bin/bash

dir="$1"
read -p "Delete all .tmp files? (y/n): " choice
if [ "$choice" = "y" ]
then
rm "$dir"/*.tmp
echo "Deleted"
else
echo "Cancelled"
fi
```

```
workspace@ubuntu:~/asgmt$ ./del_temp.sh .
Delete all .tmp files? (y/n): y
Deleted
```

67. Write a script that renames all .txt files in the current directory by changing their extension to .bak.

```
#!/bin/bash

for file in *.txt
do
mv "$file" "${file%.txt}.bak"
done
```

```
workspace@ubuntu:~/asgmt$ nano rename.sh
workspace@ubuntu:~/asgmt$ chmod a+x rename.sh
workspace@ubuntu:~/asgmt$ cd backup
workspace@ubuntu:~/asgmt/backup$ ../../rename.sh
workspace@ubuntu:~/asgmt/backup$ ls
notes.bak  notes2.bak  numbers.bak  repeated.bak  test.bak
```

68. Write a script that reads a list of numbers from a file and calculates their sum using a loop.

```
#!/bin/bash

sum=0

while read number
do
sum=$((sum+number))
done < "$1"

echo "Sum: $sum"
```

```
workspace@ubuntu:~/asgmt$ cat numbers.txt
1
2
3
4
5
6
7
8
9
10
```

```
workspace@ubuntu:~/asgmt$ nano list_sum.sh
workspace@ubuntu:~/asgmt$ ./list_sum.sh numbers.txt
Sum: 55
```

69. Write a script that reads a file containing student marks and prints only students whose marks are greater than 75.

```
#!/bin/bash

while read name marks
do
if [ $marks -gt 75 ]
then
echo "$name $marks"
fi
done < "$1"
```

```
workspace@ubuntu:~/asgmt$ cat >> marks.sh << EOD
> Rahul 80
Amit 65
Priya 90
Neha 72
> EOD
workspace@ubuntu:~/asgmt$ nano pass_students.sh
workspace@ubuntu:~/asgmt$ chmod a+x pass_students.sh
workspace@ubuntu:~/asgmt$ ./pass_students.sh marks.sh
Rahul 80
Priya 90
workspace@ubuntu:~/asgmt$
```

70. Write a script that accepts a username and checks whether that user exists in /etc/passwd using grep.

```
#!/bin/bash

read -p "Enter username: " username

if grep -q "^$username:" /etc/passwd
then
echo "User exists"
else
echo "User not exists"
fi
```

```
workspace@ubuntu:~/asgmt$ nano check_user.sh
workspace@ubuntu:~/asgmt$ ./check_user.sh
Enter username: workspace
User exists
workspace@ubuntu:~/asgmt$
```

71. Write a script that displays the current date, current user, current working directory, and number of running processes.

```
#!/bin/bash

echo "Date: $(date)"
echo "User: $USER"
echo "Directory: $(pwd)"
echo "Running Processes: $(ps aux | tail -n +2 | wc -l)"
```

```
workspace@ubuntu:~/asgmt$ nano details.sh
workspace@ubuntu:~/asgmt$ chmod a+x details.sh
workspace@ubuntu:~/asgmt$ ./details.sh
Date: Mon Sep 14 02:31:05 AM IST 2026
User: workspace
Directory: /home/workspace/asgmt
Running Processes: 310
workspace@ubuntu:~/asgmt$
```

72. Write a script that repeatedly displays a simple menu until the user selects Exit.

```
#!/bin/bash

while true
do
    echo "1. Create File"
    echo "2. Delete File"
    echo "3. Display File"
    echo "4. Exit"

    read -p "Enter choice: " choice

    case $choice in
        1)
            read -p "Filename: " file
            touch "$file"
            ;;
        2)
            read -p "Filename: " file
            rm "$file"
            ;;
        3)
            read -p "Filename: " file
            cat "$file"
            ;;
        4)
            echo "Exiting"
            break
            ;;
        *)
            echo "Invalid choice"
            ;;
    esac
done
```

```
workspace@ubuntu:~/asgmt$ nano display.sh
workspace@ubuntu:~/asgmt$ chmod a+x display.sh
workspace@ubuntu:~/asgmt$ ./display.sh
1. Create File
2. Delete File
3. Display File
4. Exit
Enter choice: 1
Filename: file.txt
1. Create File
2. Delete File
3. Display File
4. Exit
Enter choice: 4
Exiting
```

73. Write a script that accepts a filename and reports whether it is empty or contains data.

```
#!/bin/bash

if [ -s "$1" ]
then
echo "File contains data"
else
echo "File is empty"
fi
```

```
workspace@ubuntu:~/asgmt$ nano check_emptyfile.sh
workspace@ubuntu:~/asgmt$ chmod a+x check_emptyfile.sh
workspace@ubuntu:~/asgmt$ ./check_emptyfile.sh file.txt
File is empty
workspace@ubuntu:~/asgmt$ ./check_emptyfile.sh marks.sh
File contains data
workspace@ubuntu:~/asgmt$
```

74. Write a shell script that monitors the disk usage of / every 10 seconds and prints a warning when usage exceeds 80%.

```
#!/bin/bash

while true
do
usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
echo "Disk usage: $usage%"

if [ $usage -gt 80 ]
then
echo "WARNING: Disk usage exceeds 80%"
fi
sleep 10
done
```

```
workspace@ubuntu:~/asgmt$ nano check_space.sh
workspace@ubuntu:~/asgmt$ chmod a+x check_space.sh
workspace@ubuntu:~/asgmt$ ./check_space.sh
Disk usage: 22%
Disk usage: 22%
```

75. Write a shell script that asks the user for a command, executes it, and prints whether the command completed successfully or returned an error.

```
#!/bin/bash

read -p "Enter Command: " command

$command

if [ $? -eq 0 ]
then
echo "Command completed successfully"
else
echo "Command returned an error"
fi
```

```
workspace@ubuntu:~/asgnmt$ nano check_status.sh
workspace@ubuntu:~/asgnmt$ ./check_status.sh
Enter Command: hello123
./check_status.sh: line 5: hello123: command not found
Command returned an error
workspace@ubuntu:~/asgnmt$ nano check_status.sh
workspace@ubuntu:~/asgnmt$ ./check_status.sh
Enter Command: echo "yo"
"yo"
Command completed successfully
workspace@ubuntu:~/asgnmt$
```